Методичні вказівки для виконання лабораторної роботи №2

Статистична корекція кольору

1. **Мета роботи**

Вивчити найчастіше використовувані моделі кольору, реалізувати програму, що дозволяє аналізувати зображення в різних колірних просторах, а також проводити статистичну корекцію кольору зображення або його обраної ділянки.

2. **Статистична корекція кольору**

Існує безліч методів корекції кольорів зображень.

Розглянемо задачу, мета якої надати цільовому зображенню колорит деякого іншого зображення так, щоб цільове зображення виглядало природним та зберегло свої особливості (композицію, контрастність, світлотінь).

Приклад:

Застосовуємо палітру з цього малюнка	до малюнка …	Отримуємо результат:
		

Метод вирішення поставленого завдання ґрунтується на використанні статистичних даних.

Суть методу

Отже, задані цільове зображення та зображення-джерело кольору. Метод полягає в тому, що насамперед обчислюються математичне очікування E та дисперсія кольору D на обох зображеннях для кожного колірного каналу (колірний простір, у якому задані зображення, спочатку не уточнюється):

$$E_I = \frac{1}{n_I}\sum_{j=1}^{n_I} C_I^j, \qquad D_I = \sqrt{\frac{1}{n_I}\sum_{j=1}^{n_I}(C_I^j - E_I)^2}$$

де $I \in \{s,t\}$;

C - даний колірний канал j-го пікселя у заданому колірному просторі;

n - кількість пікселів цього зображення;

E - математичне очікування;

D - дисперсія;

індекси s, t означають належність відповідно до зображення - джерела кольору (source) і цільового зображення (target).

Алгоритм полягає в тому, що до кожного каналу кожного пікселя цільового зображення застосовується таке перетворення:

$$C_t^{new} = E_s + (C_t - E_t)\frac{D_s}{D_t},$$

Після застосування алгоритму математичне очікування кольору цільового зображення стає рівним математичному очікуванню зображення-джерела кольору. Аналогічно змінюється дисперсія. Або, іншими словами, гістограма цільового зображення зсувається та розтягується.

Гістограма – це графік розподілу тонів зображення. На горизонтальній осі – шкала яскравостей тонів (наприклад, від білого до чорного), на вертикальній осі – кількість пікселів заданої яскравості (частота появи кольору на зображенні).

Алгоритм сам по собі простий і може виконуватися в будь-якому колірному просторі, але результат його роботи дуже залежить від вибору простору.

Вибір колірного простору

Отже, чому не варто використовувати довільний колірний простір і як вибрати підходяще?
Насамперед, хотілося б визначити загальні вимоги до робочого кольорового простору.
Наступний список є розумним:

- Рівномірність - це вимога означає, що за однакових змін чисельного значення кольору однаково змінюється і зорове сприйняття кольору. Це важливо у цьому методі, бо значення кольорів змінюються пропорційно (див. формулу перетворення кожного каналу).
- Низький рівень кореляції між каналами - забезпечується максимально мала можливість виникнення артефактів при зміні значення одного каналу.

Було показано, що застосування алгоритму часто використовуваних колірних просторах (RGB, CMYK та ін.) не дає хороших результатів, бо ці простори не відповідають переліченим вимогам.

Простір Ruderman Lab (далі Lab) , св'язок з RGB

Оскільки зображення в переважній більшості випадків зберігається у форматі RGB, розглянемо перехід з RGB до Lab і назад.

Перехід з RGB до Lab:

Крок 1. Перехід з RGB у речовому нормуванні (тобто. всі компоненти змінюються від 0 до 1) в конічний простір LMS, бо lab - його модифікація:

$$\begin{bmatrix} L \\ M \\ S \end{bmatrix} = \begin{bmatrix} 0.3811 & 0.5783 & 0.0402 \\ 0.1967 & 0.7244 & 0.0782 \\ 0.0241 & 0.1288 & 0.8444 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

Крок 2. Перехід LMS до Lab:

$$\begin{bmatrix} l \\ a \\ b \end{bmatrix} = \begin{bmatrix} 0.5774 & 0 & 0 \\ 0 & 0.4082 & 0 \\ 0 & 0 & 0.7071 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & -2 \\ 1 & -1 & 0 \end{bmatrix} \begin{bmatrix} \lg L \\ \lg M \\ \lg S \end{bmatrix}$$

Перехід з Lab до RGB:

Крок 1. Перехід з Lab назад до LMS.

$$\begin{bmatrix} L' \\ M' \\ S' \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & -1 \\ 1 & -2 & 0 \end{bmatrix} \begin{bmatrix} 0.5774 & 0 & 0 \\ 0 & 0.4082 & 0 \\ 0 & 0 & 0.7071 \end{bmatrix} \begin{bmatrix} l \\ a \\ b \end{bmatrix}$$

$$L = 10^{L'},\ M = 10^{M'},\ S = 10^{S'}$$

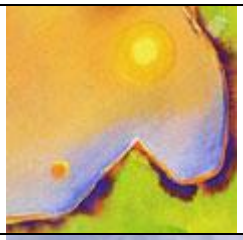
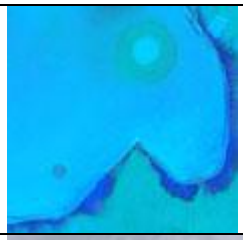
Крок 2. Перехід з LMS до RGB.

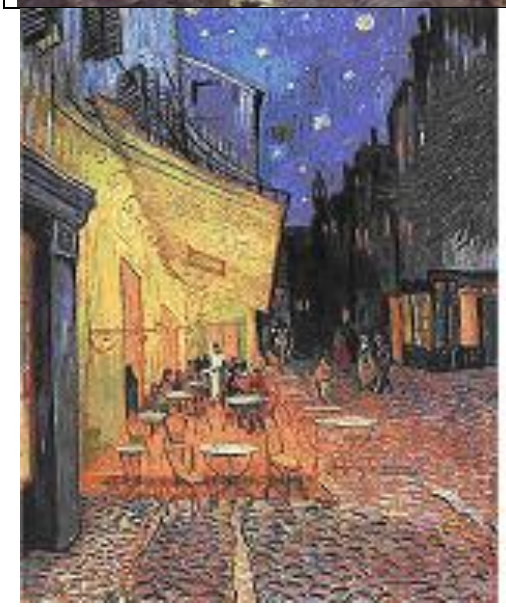
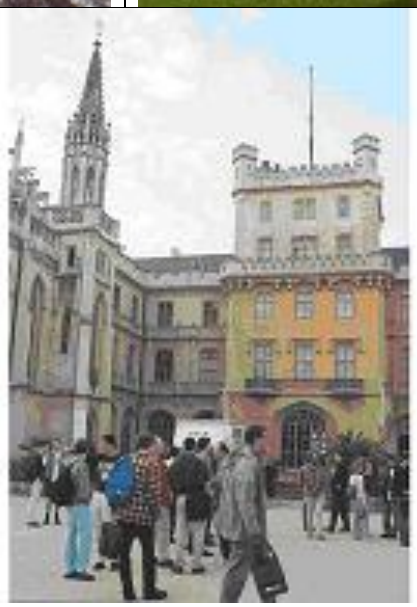
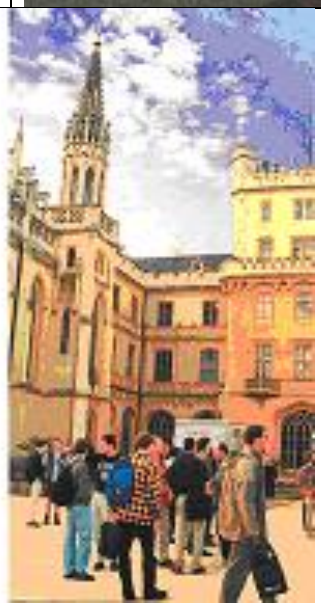
$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 4.4679 & -3.5873 & 0.1193 \\ -1.2186 & 2.3809 & -0.1624 \\ 0.0497 & -0.2439 & 1.2045 \end{bmatrix} \begin{bmatrix} L \\ M \\ S \end{bmatrix}$$

Зауваження:

- Зверніть увагу, що у формулах перекладу простір RGB встановлено в речовому нормуванні, тобто кожен компонент змінюється від 0 до 1.
- У формулах перекладу RGB в Lab є логарифмування, тому потрібно виключити 0 з області значень компонент, наприклад, вважати мінімально допустимим значенням 0.01176 (тобто 3/255).
- Щоб уникнути небажаних артефактів (вихід з області допустимих значень) при отриманні результуючого RGB-зображення, потрібно перед переходом з RGB в Lab стиснути всі компоненти так, щоб білому відповідало значення близько 0.92157 (235/255), тобто просто помножити на відповідне число. А перейшовши з Lab у RGB відновити початковий масштаб.

3. **Конкретні приклади фарбування**

Застосовуємо палітру з цього малюнка	до малюнку …	Отримуємо результат:
		
		

4. **Вимоги до програми**

Обидва зображення повинні задаватися користувачем і позначатись так, щоб було видно, де джерело, а де цільове зображення. Повинна бути можливість одночасно побачити джерело кольору, цільове зображення та результат перефарбування. Обов'язково мають бути враховані особливості, описані у зауваженнях.

Обов'язкова частина:

- Завантаження та відображення BMP-файлів (достатньо працювати з непалітровим форматом – 24-бітні зображення).
- Реалізація описаного алгоритму у просторі Lab.
- Відображення зображення в кожному з каналів кольору окремо.
- Побудова гістограм кожного кольорового каналу для всіх трьох зображень: «джерело», «мета» та «результат»

5. **Зміст звіту**
   - Постановка задачі – опис алгоритму перефарбування та алгоритму переведення в інший колірний простір (беремо з умови лабораторної);
   - Ілюстрації, що показують роботу програми;
   - Опис додаткових можливостей програми (реалізовано декілька форматів, можливість змінювати канали робочого кольору як одночасно, так і окремо тощо).

6. **Розподіл балів**

Термін здачі лабораторної: ***12.10.26***

- Реалізація основної частини: **8 баллів**
  - Завантаження зображення
  - Відображення картинки в кожному з каналів кольору колірного простору
  - Побудова гістограми для кожного кольору для всіх трьох зображень
  - Перехід з RGB простору до Lab простору
  - Статистична корекція кольору
- Наявність звіту з лабораторної роботи: **2 балла**
- 2 додаткові простори з HSL, HSV, YIQ **+1 балл**
- Можливість застосовувати корекцію кольору як одночасно до всіх колірних каналів, так і до заданих каналів **+1 балл**
- Можливість робити корекцію кольорів на заданій ділянці (прямокутний **та** круговий пензлик) **+2 балла**
- Вдалий інтерфейс та додаткова функціональність: **+2 балла**

7. **Додатки**

   a. **Простір HSL, зв’язок з RGB**

```
/*
  Calculate HSL from RGB
  Hue is in degrees
  Lightness is between 0 and 1
  Saturation is between 0 and 1
*/
HSL RGB2HSL(COLOUR c1)
{
  double themin,themax,delta;
  HSL c2;
  themin = MIN(c1.r,MIN(c1.g,c1.b));
  themax = MAX(c1.r,MAX(c1.g,c1.b));
  delta = themax - themin;
  c2.l = (themin + themax) / 2;
  c2.s = 0;
  if (c2.l > 0 && c2.l < 1)
    c2.s = delta / (c2.l < 0.5 ? (2*c2.l) : (2-2*c2.l));
  c2.h = 0;
```

```
  if (delta > 0) {
    if (themax == c1.r && themax != c1.g)        c2.h += (c1.g - c1.b) / delta;
    if (themax == c1.g && themax != c1.b)        c2.h += (2 + (c1.b - c1.r) / delta);
    if (themax == c1.b && themax != c1.r)        c2.h += (4 + (c1.r - c1.g) / delta);
    c2.h *= 60;
  }
  return(c2);
}
```

```
/*
  Calculate RGB from HSL, reverse of RGB2HSL()
  Hue is in degrees
  Lightness is between 0 and 1
  Saturation is between 0 and 1
*/
COLOUR HSL2RGB(HSL c1)
{
  COLOUR c2,sat,ctmp;
  while (c1.h < 0)      c1.h += 360;
  while (c1.h > 360)  c1.h -= 360;
  if (c1.h < 120) {
    sat.r = (120 - c1.h) / 60.0;
    sat.g = c1.h / 60.0;
    sat.b = 0;
  } else if (c1.h < 240) {
    sat.r = 0;
    sat.g = (240 - c1.h) / 60.0;
    sat.b = (c1.h - 120) / 60.0;
  } else {
    sat.r = (c1.h - 240) / 60.0;
    sat.g = 0;
    sat.b = (360 - c1.h) / 60.0;
  }
  sat.r = MIN(sat.r,1);
  sat.g = MIN(sat.g,1);
  sat.b = MIN(sat.b,1);
  ctmp.r = 2 * c1.s * sat.r + (1 - c1.s);
  ctmp.g = 2 * c1.s * sat.g + (1 - c1.s);
  ctmp.b = 2 * c1.s * sat.b + (1 - c1.s);
  if (c1.l < 0.5) {
    c2.r = c1.l * ctmp.r;
    c2.g = c1.l * ctmp.g;
    c2.b = c1.l * ctmp.b;
  } else {
    c2.r = (1 - c1.l) * ctmp.r + 2 * c1.l - 1;
    c2.g = (1 - c1.l) * ctmp.g + 2 * c1.l - 1;
    c2.b = (1 - c1.l) * ctmp.b + 2 * c1.l - 1;
  }
  return(c2);
}
```

b. Простір HSV, зв’язок з RGB

```
/*
  Calculate RGB from HSV, reverse of RGB2HSV()
  Hue is in degrees
  Lightness is between 0 and 1
  Saturation is between 0 and 1
*/
COLOUR HSV2RGB(HSV c1)
```

```
{
  COLOUR c2,sat;
  while (c1.h < 0)      c1.h += 360;
  while (c1.h > 360)  c1.h -= 360;
  if (c1.h < 120) {
    sat.r = (120 - c1.h) / 60.0;
    sat.g = c1.h / 60.0;
    sat.b = 0;
  } else if (c1.h < 240) {
    sat.r = 0;
    sat.g = (240 - c1.h) / 60.0;
    sat.b = (c1.h - 120) / 60.0;
  } else {
    sat.r = (c1.h - 240) / 60.0;
    sat.g = 0;
    sat.b = (360 - c1.h) / 60.0;
  }
  sat.r = MIN(sat.r,1);
  sat.g = MIN(sat.g,1);
  sat.b = MIN(sat.b,1);
  c2.r = (1 - c1.s + c1.s * sat.r) * c1.v;
  c2.g = (1 - c1.s + c1.s * sat.g) * c1.v;
  c2.b = (1 - c1.s + c1.s * sat.b) * c1.v;
  return(c2);
}


/*
  Calculate HSV from RGB
  Hue is in degrees
  Lightness is betweeen 0 and 1
  Saturation is between 0 and 1
*/
HSV RGB2HSV(COLOUR c1)
{
  double themin,themax,delta;
  HSV c2;
  themin = MIN(c1.r,MIN(c1.g,c1.b));
  themax = MAX(c1.r,MAX(c1.g,c1.b));
  delta = themax - themin;
  c2.v = themax;
  c2.s = 0;
  if (themax > 0)      c2.s = delta / themax;
  c2.h = 0;
  if (delta > 0) {
    if (themax == c1.r && themax != c1.g)        c2.h += (c1.g - c1.b) / delta;
    if (themax == c1.g && themax != c1.b)       c2.h += (2 + (c1.b - c1.r) / delta);
    if (themax == c1.b && themax != c1.r)       c2.h += (4 + (c1.r - c1.g) / delta);
    c2.h *= 60;
  }
  return(c2);
}
```

c. Простір YIQ, зв’язок з RGB

```
Y = 0.299*R  +  0.587*G  +  0.114*B
I  =  0.596*R  -  0.275*G   -  0.321*B
Q = 0.212*R  -  0.528*G   +  0.311*B

R =  Y  +  0.956*I  +  0.621*Q
G =  Y  -  0.272*I   -  0.647*Q
B =  Y  -  1.106*I   -  1.703*Q
```